

# Base neural network + PPO

DAVIDE FIORENTINI

November 2025

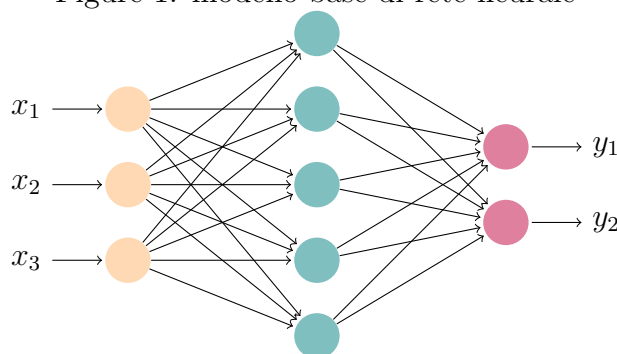
## 1 introduzione

in questo documento saranno elencati i processi matematici con cui vengono effettuate le operazioni di *front-propagation* e di *back-propagation* in una rete neurale, allenata usando il *PPO*.

## 2 front-propagation

per comprendere la fase di *front-propagation* dobbiamo visualizzare la struttura di una rete neurale

Figure 1: modello base di rete neurale



### 2.1 modello

Come visto nella figura 1 la struttura si divide in diverse *layer*: input, intermedi e di output. nella fase di front-propagation vogliamo far passare i vari

input attraverso i layer intermedi per trovare la probabilità di ogni azione rappresentata dai layer di output. Per fare questa transizione dobbiamo vedere come lavora ogni singolo neurone nel modello.

## 2.2 funzionamento di un neurone

Ogni neurone si può suddividere in 2 fasi:

### 2.2.1 calcolo della funzione di trasferimento dei pesi

in questa fase il neurone compie il calcolo:

$$Z_n = \sum_{j=1}^{N_{l_{i-1}}} (W_{nj} * \sigma[j]_{l_{i-1}}) + B_n \quad (1)$$

con  $K$  che rappresenta il numero di nodi al layer  $i - 1$ , il peso  $W_{nj}$  tra in nodo  $n$  e il nodo al layer  $i - 1$   $j$ , la funzione di attivazione nel nodo  $j$   $\sigma$  e  $B$  il bias del nodo  $n$ .

### 2.2.2 calcolo del livello di attivazione del neurone

In questa fase lo  $Z_n$  trovato dall'equazione 1 verrà inserito in una funzione di attivazione  $\sigma$  (generalmente ReLu ma c'è ne sono di diversi tipi).

$$\sigma(z) = Relu(z) = max(0, z) \quad (2)$$

oppure

$$\sigma(z_i) = SoftMax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}} \quad for \ i = 1, 2, \dots, n \quad (3)$$

con in generale l'equazione 2 usata per l'attivazione dei nodi intermedi e la 3 per ottenere un vettore di probabilità dai nodi finali  $n$ .

con questo abbiamo finito di vedere la front-propagation.

## 3 Back-Propagation

La back-propagation consiste nel cercare la quantità  $\Delta$  (step-size) con cui variare ogni parametro  $W$  e  $B$  per far migliorare i risultati. Per fare questo si passa attraverso 3 fasi.

### 3.1 Loss-function

La loss-function (L) è quella funzione che la rete deve minimizzare o massimizzare, di solito ha una forma del tipo:

$$L = \sum_{j=1}^n (\sigma[j] - Y)^2 \quad (4)$$

oppure

$$L = - \sum_{j=1}^n (P(j) * \ln(P(j))) \quad (5)$$

o di varie altre forme, l'importante è avere una funzione che ti indichi quanto la rete sia andata bene/male.

Un ultima cosa da sapere è che normalmente si calcola  $E[L]$  intesa come la stima della  $L$ :

$$E[L] = \frac{1}{N} * \sum_{i=1}^N L_i \quad (6)$$

con  $N$  che rappresenta il numero di campioni in un batch, poi tutti i calcoli successivi di derivazioni si compiono sulla  $E[L]$  (quindi quando vedete  $dL$  dovete pensare alla derivata di  $E[L]$ , ma dato che è un'operazione lineare  $dE[L] = E[dL]$ ).

### 3.2 Calcolo delle derivate

In questa fase cerchiamo in che direzione devono variare i parametri  $W$  e  $B$  di ogni neurone, perciò dobbiamo calcolare la derivata  $\frac{dL}{dW}$  e  $\frac{dL}{dB}$  per ogni parametro usando la chain-rule. Quindi consideriamo la derivata  $\frac{dL}{d\sigma}$ :

$$\frac{dL}{d\sigma[n]_{l_t}} = \sum_{j=1}^n \frac{dZ[j]_{l_{t+1}}}{d\sigma[n]_{l_t}} * \frac{d\sigma[j]_{l_{t+1}}}{dZ[j]_{l_{t+1}}} * \frac{dL}{d\sigma[j]_{l_{t+1}}} \quad (7)$$

Considerando  $0 \leq t \leq k$  il layer.

Allora la derivata dei vari parametri  $W$  e  $B$  sarà:

$$\frac{dL}{dW_{ji}} = \frac{dL}{d\sigma[i]_{l_t}} * \frac{d\sigma[i]_{l_t}}{dZ} * \frac{dZ}{dW_{ji}} \quad (8)$$

$$\frac{dL}{dB_n} = \frac{dL}{d\sigma[i]_{l_t}} * \frac{d\sigma[i]_{l_t}}{dZ} * \frac{dZ}{dB_n} \quad (9)$$

### 3.3 Step-Size

Calcolata la derivata del generico parametro  $\theta$  (W e B) ora dobbiamo variare i parametri per minimizzare l'errore.

$$\Delta_{\theta} = \alpha_{\theta} * \frac{dL}{\theta} \quad (10)$$

Al parametro quindi:

$$\theta_{new} = \theta_{old} \pm \Delta_{\theta} \quad (11)$$

in cui all'equazione 11 si somma l'equazione 10 se si è nel caso di *gradient ascend*, se invece si deve fare il *gradient descent* si sottrae.

Ora vedremo il caso specifico del *PPO*.

## 4 Proximal Policy Optimization (PPO)

Nel PPO si usano 2 reti neurali, una da actor (che compie azioni) e una da critic (che valuta le azioni della prima). Entrambe avranno come ingresso lo stato  $S$ , l'actor darà come output un'azione  $a$ , il critico darà un valore di valutazione  $V$ .

Ora vedremo le caratteristiche dei 2.

### 4.1 Actor

L'actor si divide principalmente in 3 passaggi:

#### 4.1.1 Calcolo Esperienza

L'actor inizia facendo delle azioni in base alla propria policy, da queste azioni accumula esperienza:

$$A_t = \sum_{l=0}^T (\gamma * \lambda)^l * \delta_{t+l} \quad (12)$$

in cui

$$\delta_t = r_t + \gamma * V(s_{t+1}) - V(s_t) \quad (13)$$

con  $\gamma$  [0.9-0.99] e  $\lambda$  [0-1] hyperparametri,  $r$  il reward e  $T$  il numero di timestamp in cui l'actor ha ottenuto esperienze dal tempo  $t$ . da questo possiamo calcolare.

### 4.1.2 Calcolo miglioramento di policy

Il miglioramento di policy si può calcolare:

$$R_t(\theta) = \frac{P_{\theta_{new}}(s_t)}{P_{\theta_{old}}(s_t)} \quad (14)$$

in cui  $P_{\theta}(s_t)$  è la probabilità di una azione con parametri  $\theta$  allo stato  $s_t$ .  
Notare che se  $R_t(\theta)$  è  $>$  di 1 l'azione è diventata più probabile, se  $<$  il contrario.

### 4.1.3 Actor Loss-function

La loss-function deriva da quello che abbiamo visto prima ed è:

$$L_t^{clip}(\theta) = \min(R_t(\theta) * A_t, \text{clip}(R_t(\theta), 1 - \varepsilon, 1 + \varepsilon) * A_t) \quad (15)$$

In cui  $\varepsilon[0.1-0.3]$  è un hyperparametro e clip è definita come:

$$\text{clip}(x, a, b) = \begin{cases} x < a & a \\ x > b & b \\ a \leq x \leq b & x \end{cases} \quad (16)$$

Poi sull'equazione 15 si aggiungerà un elemento di entropia per permettere all'actor di esplorare in maniera più minuziosa l'ambiente.

$$L_t^{actor}(\theta) = L_t^{clip}(\theta) + c_1 * CE(\theta) \quad (17)$$

In cui CE deriva dall'equazione 5 e  $c_1 [0.1-1]$  è un hyperparametro.

L'actor utilizza il Gradient Ascend nella fase di back-propagation, come visto dall'equazione 11.

## 4.2 Critic

Il critic calcola semplicemente la funzione  $V(s_t)$  che abbiamo visto usata dall'actor. Quindi la rete neurale dovrà approssimare V usando il Gradient Descent alla funzione:

$$L^{eval} = (V_{\theta}(s_t) - V_t^{target})^2 \quad (18)$$

con

$$V_t^{target} = \sum_{l=0}^T \gamma^l * r_{t+l} \quad (19)$$

## 5 conclusioni

Notare che la preparazione dei batch di esperienza e la fase di gradient update possono essere fatti in parallelo. Si noti anche che alcuni layer iniziali dell'actor e critic possono essere condivisi, basta fare una nuova loss-function che consideri le loro loss-function.

In step sarebbe:

1. Gli actor esplorano il mondo fino a un tempo  $T$  o terminano, il critic valuta le performance mentre la vecchia policy viene comparata con la nuova, tutto in parallelo.
2. Usando il calcolo sulla vecchia policy viene calcolata la  $L$  e viene effettuata la back-propagation.
3. Si riparte dal punto 1